

Sprint 1 - TestCommerce

Integrantes:

- Escalante, Nahuel
- Bustamante, Enrique
- Silva, Maximo
- Azcoaga, Christian
- Franco, Tomas

Roles:

- Project Manager: Bustamante, Enrique
- Developer 1: Azcoaga, Christian
- Developer 2: Silva, Maximo
- Tester 1: Escalante, Nahuel
- Tester 2: Franco, Tomas

Objetivo del sprint

El objetivo del Sprint 1 fue comprender el sistema base **Test Commerce**, definir el alcance funcional del proyecto, organizar los roles del equipo y preparar la documentación inicial para comenzar con las actividades de testing.

El sistema elegido es una web de prueba tipo e-commerce, desarrollada para aplicar testing de aplicaciones sobre funcionalidades como login, catálogo de productos, carrito de compras, checkout, validación de stock, roles de usuario y endpoints.

Actividades realizadas

Durante el Sprint 1 se revisó el proyecto base desarrollado en Python con Flask.

Se analizó la estructura general del sistema y se identificaron las funcionalidades principales incluidas.

Se definió que el proyecto no busca ser un e-commerce real terminado, sino una aplicación base para ejecutar pruebas funcionales, pruebas de endpoints, automatización y pruebas no funcionales.

También se establecieron los roles del equipo, se refinó el alcance funcional del sistema, se definieron requerimientos iniciales, reglas de negocio, backlog inicial y un primer plan de pruebas.

Funcionalidades relevadas

Página de inicio.

Catálogo de productos.

Detalle de productos.

Carrito de compras.

Agregar productos al carrito.

Eliminar productos del carrito.

Modificación de cantidades en el carrito.

Cálculo de subtotal y total.

Login de usuario común.

Login de administrador.

Cierre de sesión.

Checkout.

Validación de stock.

Panel administrador.

Alta, edición y eliminación de productos.

Persistencia mediante archivo JSON.

Endpoints JSON para operaciones del carrito.

Requerimientos funcionales iniciales

- RF01:** El sistema debe permitir visualizar un catálogo de productos.
 - RF02:** El sistema debe permitir ver el detalle de un producto.
 - RF03:** El sistema debe permitir agregar productos al carrito.
 - RF04:** El sistema debe permitir eliminar productos del carrito.
 - RF05:** El sistema debe permitir modificar la cantidad de productos en el carrito.
 - RF06:** El sistema debe calcular el subtotal por producto y el total general del carrito.
 - RF07:** El sistema debe permitir iniciar sesión como usuario común.
 - RF08:** El sistema debe permitir iniciar sesión como administrador.
 - RF09:** El sistema debe permitir cerrar sesión.
 - RF10:** El sistema debe permitir finalizar una compra mediante checkout.
 - RF11:** El sistema debe validar que el usuario esté logueado antes de finalizar una compra.
 - RF12:** El sistema debe impedir agregar productos sin stock.
 - RF13:** El sistema debe permitir al administrador crear productos.
 - RF14:** El sistema debe permitir al administrador editar productos.
 - RF15:** El sistema debe permitir al administrador eliminar productos.
 - RF16:** El sistema debe guardar la información de productos mediante archivos JSON.
 - RF17:** El sistema debe exponer endpoints para consultar y modificar el carrito.
-

Reglas de negocio iniciales

- Un usuario no logueado no puede finalizar una compra.
- Un producto sin stock no debe poder agregarse al carrito.
- El carrito debe mostrar correctamente la cantidad, el subtotal y el total general.

El checkout no debe confirmarse si existen campos obligatorios vacíos.

Solo un usuario administrador puede acceder al panel de administración.

Los productos deben tener nombre, precio, categoría y stock.

El stock no puede ser negativo.

El precio de un producto no puede ser negativo.

Las operaciones principales deben poder validarse mediante pruebas manuales y pruebas de endpoints.

Backlog inicial

ID	Tarea	Prioridad	Responsable
B01	Revisar estructura del proyecto base	Alta	Developer
B02	Documentar funcionalidades existentes	Alta	Project Manager
B03	Definir requerimientos funcionales	Alta	Project Manager
B04	Definir reglas de negocio	Alta	Project Manager / Testers
B05	Identificar módulos a testear	Alta	Testers
B06	Diseñar plan de pruebas inicial	Alta	Testers

B07	Relevar endpoints existentes	Alta	Developer
B08	Definir endpoints faltantes o nuevos	Media	Developer / Testers
B09	Definir datos de prueba	Media	Testers
B10	Revisar posibles defectos o validaciones débiles	Media	Testers
B11	Preparar documentación inicial	Alta	Project Manager
B12	Organizar evidencias por sprint	Media	Todo el equipo

Primer plan de pruebas

Objetivo

Validar que las funcionalidades principales del sistema TestCommerce funcionen correctamente y permitan aplicar pruebas manuales, pruebas de endpoints, automatización con Selenium y pruebas no funcionales con JMeter.

Alcance inicial

Se probarán los siguientes módulos:

Login.

Catálogo de productos.

Carrito de compras.

Checkout.

Validación de stock.

Panel administrador.

Endpoints del carrito.

Tipos de prueba previstos

Pruebas funcionales manuales.

Pruebas de validación.

Pruebas de endpoints.

Pruebas de regresión.

Automatización con Selenium.

Pruebas no funcionales con JMeter.

Herramientas previstas

Navegador web.

Postman o Thunder Client.

Selenium.

JMeter.

GitHub.

Capturas de pantalla como evidencia.

Planilla para casos de prueba y defectos.

Datos de prueba iniciales

Usuario común

Email: usuario@test.com

Contraseña: 1234

Usuario administrador

Email: admin@test.com

Contraseña: 1234

Producto con stock

Producto: iPhone 15

Condición: stock mayor a 0

Producto sin stock

Producto: Notebook Lenovo

Condición: stock igual a 0

Datos válidos de checkout

Nombre: Usuario Prueba

Dirección: Av. Corrientes 1234

Teléfono: 1123456789

Método de pago: MercadoPago

Datos inválidos

Email incorrecto.

Contraseña incorrecta.

Campos vacíos.

Cantidad mayor al stock.

Producto sin stock.

Usuario no logueado.

Sprint	Tarea	Responsable	Duración	Fechas	Descripción simple	Entregable
S1	Setup ambiente	Dev 1 + Dev 2	2 días	22-23/05	Clonar repo, instalar dependencias y levantar la app localmente.	App corriendo en http://127.0.0.1:5000
S1	Documentar estructura app.py	Dev 1 + Dev 2	2 días	22-23/05	Entender el código, rutas, sesiones y funcionamiento del carrito.	Documento de arquitectura
S1	Diseñar 2 endpoints JSON	PM	2 días	23-24/05	Especificar los endpoints GET /api/productos y POST /api/pedidos.	Documento de especificación
S1	Implementar GET /api/productos	Dev 1 + Dev 2	3 días	24-26/05	Implementar filtros por categoría, precio, disponibilidad, búsqueda y ordenamiento.	Endpoint funcional
S1	Implementar POST /api/pedidos	Dev 1 + Dev 2	4 días	25-28/05	Agregar validaciones, verificar stock, generar ID de pedido y devolver respuesta JSON.	Endpoint funcional
S1	Tests unitarios endpoints	Dev 1 + Dev 2	2 días	27-28/05	Crear tests para validaciones y lógica de filtrado.	Suite de tests unitarios

S1	Analizar funcionalidades existentes	Tester 1 + Tester 2	4 días	22-25/05	Probar catálogo, carrito, login y panel admin.	Documento de funcionalidades base
S1	Diseñar casos de prueba	Tester 1 + Tester 2	6 días	22-27/05	Diseñar más de 30 casos para endpoints, carrito, login y admin.	Plan de pruebas completo
S1	Matriz de trazabilidad	Tester 1 + PM	2 días	28-29/05	Mapear qué caso de prueba cubre cada requerimiento.	Matriz de trazabilidad
S2	Ejecutar pruebas manuales	Tester 1 + Tester 2	6 días	05-10/06	Navegar la app, completar formularios y verificar totales.	Evidencia con screenshots
S2	Pruebas de API con Postman/curl	Tester 1 + Tester 2	4 días	05-08/06	Testear endpoints JSON y validar respuestas correctas y errores.	Reportes de pruebas API
S2	Registrar defectos encontrados	Tester 1 + Tester 2	3 días	06-08/06	Documentar bugs con pasos, resultado esperado y resultado obtenido.	Registro de defectos
S2	Corregir defectos críticos	Dev 1 + Dev 2	3 días	09-11/06	Arreglar bugs y reforzar validaciones.	Código corregido

S2	Pruebas de regresión	Tester 1 + Tester 2	4 días	09-12/06	Re-ejecutar casos después de las correcciones.	Evidencia de regresión
S2	Agregar logging y mejoras	Dev 1 + Dev 2	2 días	10-11/06	Agregar logs en operaciones clave y limpiar el código.	Código con logging
S3	Diseñar escenarios Selenium	Tester 1 + Tester 2	2 días	19-20/06	Seleccionar más de 3 flujos, como login, carrito y checkout.	Documento de escenarios
S3	Automatizar con Selenium	Tester 1 + Tester 2	5 días	20-24/06	Escribir scripts en Python con Selenium y assertions.	Suite de tests Selenium
S3	Ejecutar tests Selenium	Tester 1 + Tester 2	2 días	24-25/06	Ejecutar scripts y capturar resultados.	Reporte de Selenium
S3	Diseñar escenarios JMeter	Tester 1 + Tester 2	2 días	19-20/06	Armar plan de carga con más de 2 endpoints y entre 10 y 50 usuarios.	Archivo .jmx configurado
S3	Ejecutar tests JMeter	Tester 1 + Tester 2	3 días	21-23/06	Lanzar carga y medir response time y throughput.	Reporte de JMeter
S3	Optimizaciones finales	Dev 1 + Dev 2	2 días	24-25/06	Refactorizar, ordenar código y optimizar si aparece algún cuello de botella.	Código optimizado

S4	Consolidar evidencias	Tester 1 + Tester 2	1 día	26/06	Recopilar screenshots, reportes y casos ejecutados.	Carpeta con evidencias
S4	Preparar presentación	PM + Tester 1	1 día	26/06	Preparar slides sobre sistema, testing, resultados y conclusiones.	Presentación completa
S4	Retrospectiva a equipo	Todos	1 día	26/06	Documentar qué funcionó, qué se puede mejorar y lecciones aprendidas.	Documento retrospectiva
S4	Presentación final y demo	Todos	1 día	26/06	Presentar ante el profesor y demostrar el funcionamiento del sistema.	Presentación realizada